

CHƯƠNG 10 PHÂN LOẠI VÀ PHÁT HIỆN BỆNH Ở THỰC VẬT

Giới thiệu Trong nông nghiệp, bệnh cây trồng luôn là một vấn đề lớn, vì chúng làm giảm chất lượng cây trồng, dẫn đến giảm năng suất. Bệnh cây trồng có nhiều ảnh hưởng khác nhau, từ các triệu chứng nhỏ đến thiệt hại nghiêm trọng trên toàn bộ diện tích cây trồng, dẫn đến chi phí tài chính cao và ảnh hưởng đáng kể đến nền kinh tế nông nghiệp, đặc biệt là ở các nước đang phát triển phụ thuộc vào một hoặc một vài loại cây trồng.

Nhiều phương pháp khác nhau đã được phát triển để phát hiện bệnh nhằm tránh những tổn thất đáng kể. Các tác nhân gây bệnh đã được xác định chính xác bằng các phương pháp phát triển trong sinh học phân tử và miễn dịch học. Tuy nhiên, đối với nhiều nông dân, những kỹ thuật này khó tiếp cận và đòi hỏi kiến thức chuyên sâu về lĩnh vực hoặc tốn nhiều thời gian và công sức để sử dụng.

Theo Tổ chức Lương thực và Nông nghiệp Liên Hợp Quốc, hầu hết các trang trại trên thế giới là nhỏ và thuộc sở hữu gia đình ở các nước phát triển. Những gia đình như vậy cung cấp lương thực cho một tỷ lệ đáng kể dân số hành tinh. Tuy nhiên, đói nghèo và mất an ninh lương thực không hiếm gặp và khả năng tiếp cận thị trường cũng như dịch vụ còn hạn chế.

Vì những lý do trên, rất nhiều công trình đã được thực hiện nhằm thiết lập các phương pháp đủ tin cậy và sẵn có cho hầu hết nông dân. Trong đó, bệnh cây trồng là một yếu tố thiết yếu góp phần làm suy giảm nghiêm trọng chất lượng và số lượng sản xuất cây trồng. Nhận dạng và phân loại bệnh cây trồng đóng vai trò quan trọng trong việc cải thiện sản xuất cây trồng và phát triển kinh tế.

Khi nhận thấy bệnh cây trồng đã trở thành một mối quan tâm lớn trong nông nghiệp như thế nào, nhiều công nghệ đã được sử dụng để phát hiện bệnh một cách dễ dàng và trí tuệ nhân tạo là một trong số đó. Ví dụ, để phát hiện và phân loại bệnh cây trồng, ở đây bằng cách sử dụng học sâu (deep learning) của Trí tuệ nhân tạo, một mô hình sẽ được tạo ra và huấn luyện dựa trên bộ dữ liệu từ Kaggle là plant village (làng thực vật). Nó có thể hữu ích cho việc xác định bệnh cây trồng theo thời gian thực. Trong Dự án, chúng ta sẽ tạo một mô hình CNN (Mạng nơ-ron tích chập) được sử dụng để phân loại và phát hiện bệnh ở khoai tây dựa trên dữ liệu mà chúng ta huấn luyện nó.

Cài đặt các Gói (Packages)

Để phát triển một mô hình hoặc dự án về phát hiện bệnh cây trồng, chúng ta sẽ yêu cầu một số thư viện của Trí tuệ nhân tạo và Python. Ở đây chúng ta đã thảo luận từng bước cài đặt và mô tả của chúng để hiểu rõ hơn.

Để cài đặt tất cả các gói thư viện, chúng ta sẽ sử dụng PIP trong Windows. Các thư viện được sử dụng để phát triển dự án này là:

- Keras: Lệnh cài đặt trên Windows: `pip install keras`
- NumPy: Lệnh cài đặt trên Windows: `pip install keras`

- Math: Lệnh cài đặt trên Windows: pip install math
- Matplotlib: Lệnh cài đặt trên Windows: pip install matplotlib
- TensorFlow: Lệnh cài đặt trên Windows: pip install tensorflow

Bộ dữ liệu (Dataset)

Ở đây chúng ta lấy một Bộ dữ liệu tên là Plant village từ Kaggle, có 15 danh mục dựa trên các bệnh cây trồng khác nhau. Chúng ta đã chọn phát hiện bệnh ở cây khoai tây, trong đó chúng ta có 3 danh mục như Bệnh cháy sớm (Early Blight), Bệnh cháy muộn (Late Blight) và khoai tây khỏe mạnh (healthy potato). Đối với mỗi lớp (class) này, chúng ta sẽ có hình ảnh làm đầu vào để huấn luyện mô hình dự đoán. Bộ dữ liệu có thể được tải về từ đây. <https://www.kaggle.com/emmarex/plantdisease>

Các bước thực hiện

Trong bước đầu tiên của việc phát hiện bệnh, trước hết chúng ta sẽ nhập (import) tất cả các thư viện cần thiết cho việc phát triển dự án. Trong quá trình phát triển dự án này, Google Research Collaboratory đã được sử dụng và có thể truy cập bằng liên kết này <http://colab.research.google.com/> . Bạn cũng có thể sử dụng Anaconda để chạy dự án này nhưng vui lòng chỉ định tất cả các đường dẫn liên quan đến bộ dữ liệu và hình ảnh kiểm tra vào dự án.

Bước 1: Đầu tiên chúng ta sẽ nhập các thư viện cơ bản cần thiết để phát triển dự án phát hiện bệnh này:

```
import numpy as np
import math
import keras
import matplotlib.pyplot as plt
from tensorflow.keras.layers import *
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from tensorflow.keras import models
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.optimizers import *
from tensorflow.keras.callbacks import LearningRateScheduler
```

Output: Using TensorFlow backend. Đây là tất cả các thư viện sẽ được yêu cầu và xử lý trong suốt dự án cho các mục đích khác nhau.

Bước 2: Có hai phương pháp mà chúng ta có thể nhập dữ liệu cho dự án Phương pháp thứ nhất:

```
!pip install kaggle
!mkdir .kaggle
(Phần này nằm ở trang 3)
```

```
import json
token = {"username":"","key":""}
with open (1/content/.kaggle/kaggle.json, 'w') as file:
    json.dump (token, file)
!chmod 600 /content/.kaggle/kaggle.json
!cp/content/.kaggle/kaggle.json /root/.kaggle/kaggle.json
!kaggle datasets download -d xabdallahali/plantvillage-dataset/
color -p/content
```

Output: Downloading plantvillage-dataset.zip to /content

99% 2.02G/2.04G [00:44<00:00, 0.6MB/s]

100% 2.04G/2.04G [00:44<00:00, 1.6MB/s]

Ở đây chúng ta tải trực tiếp bộ dữ liệu từ Kaggle, bạn có thể sử dụng nó như trong tệp của mình hoặc bạn có thể sử dụng phương pháp thứ hai.

Phương pháp thứ hai: Nếu bạn đang sử dụng Google Research Collaboratory thì bạn có thể tải bộ dữ liệu từ liên kết này <https://www.kaggle.com/emmarex/plantdisease> và tải nó lên thư mục “Colab Notebooks” trên Google Drive của bạn, sau đó bạn có thể liên kết nó bằng cách sử dụng mã này

```
from google.colab import drive
drive.mount ('/gdrive')
```

Output:

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=94.....) Enter your authorization code:

Mounted at /gdrive

Đây là mã để gắn (mount) drive của bạn với Google Research Collaboratory, nó sẽ yêu cầu bạn xác thực mà bạn có thể thực hiện bằng tài khoản Gmail của mình chỉ bằng cách làm theo các bước đơn giản. Như bạn có thể thấy trong output, Collaboratory sẽ yêu cầu bạn xác minh bằng cách nhấp vào liên kết đã cho, sau đó nó sẽ được gắn bằng cách cấp quyền xác thực cho nó.

```
with open ('/gdrive/My Drive/foo.txt', 'w') as f:
    f.write ('Hello Google Drive!')
!cat '/gdrive/My Drive/foo.txt'
```

Output:

Hello Google Drive!

Đây là mã sẽ giúp bạn kiểm tra xem drive của bạn đã được gắn với Collaboratory hay chưa.

Sau khi thực hiện việc này, chúng ta sẽ truy cập bộ dữ liệu đã được tải lên trong drive trực tiếp và giải nén nó để truy cập các tệp mà chúng ta yêu cầu trong dự án này.

```
from zipfile import ZipFile
# specifying the zip file name
file name = "plantvillage-dataset.zip" #path for location of
dataset
# opening the zip file in READ mode
with ZipFile (file name, 'r') as zi:
# extracting all the files
print ('Extracting all the files now...')
zi.extractall ()
print ('Done!')
```

ouput:

Extracting all the files now...

Done!

Ở đây chúng ta chỉ đơn giản là giải nén tất cả dữ liệu từ thư mục ZIP của PlantVillage để sử dụng sau này.

#splitting data into train and test (chia dữ liệu thành tập huấn luyện và tập kiểm tra)

!pip install split-folders tqdm

Output:

Collecting split-folders Downloading https://files.pythonhosted.org/packages/20/67/29dda743e6d23ac1ea3d16704d8bbb48d65faf3f1b1eaf53153b3da56c56/split_folders-0.3.1-py3-none-any.whl Requirement already satisfied: tqdm in /usr/local/lib/python3.6/dist-packages (4.38.0) Installing collected packages: split-folders Successfully installed split-folders-0.3.1

!mkdir potato

! cp -r '/content/plantvillage dataset/segmented/Potato ___ Early _ blight' 'potato/early blight'

! cp -r '/content/plantvillage dataset/segmented/Potato ___ Late _ blight' 'potato/late blight'

! cp -r '/content/plantvillage dataset/segmented/Potato ___ healthy' 'potato/healthy'

Ở đây chúng ta đang tạo các thư mục để phát hiện bệnh khoai tây trong thư mục đã được giải nén từ bộ dữ liệu plant village.

```
import split folders
# Split with a ratio. (Chia theo tỷ lệ)
```

```
# To only split into training and validation set, set a tuple
to 'ratio', ie, '(.8, .2)'. (Để chỉ chia thành tập huấn luyện và tập xác thực, hãy đặt một bộ giá trị
cho 'ratio', ví dụ: '(.8, .2)')
#split folders.ratio ('color', output="dataset", seed $=1337$,
ratio= (.8, 1, 1)) # default values (giá trị mặc định)
split folders.ratio ('potato', output="dataset", seed $l=133^{ - }$ ,
ratio (.8, 0.2)) # default values (giá trị mặc định)
```

Output:

Copying files: 2152 files [00:00, 9225.64 files/s]

Bước 3: Xây dựng mô hình CNN Đầu tiên, chúng ta sẽ xây dựng Mô hình CNN của mình, mã nguồn như sau.

```
def model _ cnn ():
    model = Sequential ([
        BatchNormalization (axis=1, input _ shape= (256,256,3)),
        Convolution2D (32,3,3, activation='relu'), BatchNormalization
        (axis=1), MaxPooling2D ( (3,3)),
        Convolution2D (64,3,3, activation='relu'), BatchNormalization
        (axis=1), MaxPooling2D ((3,3)),
        Flatten (),
        Dense (200, activation='relu'),
        Dropout (0.2), BatchNormalization (),
        Dense (3, activation='softmax')
    ])
    return model
```

Loại mô hình mà chúng ta đang sử dụng là cơ bản và dễ dàng để tạo bất kỳ mô hình CNN nào, đó là Sequential. Ở đây chúng ta đã định nghĩa toàn bộ mô hình thành một hàm để có thể gọi nó bất cứ khi nào cần thiết. Và Sequential là phương pháp dễ nhất để xây dựng nó bằng Keras. Nó cho phép chúng ta xây dựng mô hình CNN từng Lớp (Layer by layer). Trong Mô hình, Lớp đầu tiên của chúng ta là Convolution2D (32,3,3, Activation='relu'), Nó xử lý các hình ảnh đầu vào và đổi số 32 Mô tả số lượng nút (nodes) trong lớp có thể thay đổi theo thời gian dựa trên kích thước bộ dữ liệu. Trong đó 3,3 trong đổi số mô tả kích thước hạt nhân (kernel size) là kích thước của ma trận bộ lọc cho lớp tích chập của chúng ta trong mô hình. Vì vậy, kích thước hạt nhân là 3 có nghĩa là chúng ta có ma trận bộ lọc 3X3 cho lớp mô hình của mình. Đổi số Activation là hàm kích hoạt cho lớp, là ReLU hoặc Rectified Liner Activation (Kích hoạt Tuyến tính Chỉnh lưu) hoạt động tốt cho các mạng nơ-ron.

Sau đó, chúng ta sẽ sử dụng Batch Normalization (axis=1) để chuẩn hóa và tái tỷ lệ dữ liệu, trong đó đổi số axis = 1 đại diện cho kênh (channel). Sau đó, chúng ta sẽ sử dụng max pooling 2D để chọn các đặc trưng tối đa được bao phủ bởi các bộ lọc. Trong lớp Tiếp theo Convolution2D (64,3,3, Activation='relu'), trong lớp này, chúng ta sẽ có 64 nút để xử lý hình ảnh đầu vào và tiếp tục chúng ta sẽ lại có cả chuẩn hóa hàng loạt (batch normalization) và gộp tối đa (max pooling). Giữa các lớp, chúng ta có lớp "flatten ()" luôn đóng vai trò là kết nối giữa lớp tích chập (convolution) và lớp dày đặc (dense). Trong mô hình, chúng ta sẽ có 2 lớp đầu ra, trong đó lớp đầu tiên là lớp đầu ra Dense (200, activation='relu'). Dense là một loại tiêu chuẩn trong mạng nơ-ron có thể được sử dụng cho nhiều mục đích. Ở đây, đổi số 200 trong lớp dense có nghĩa là chúng ta sẽ có 200 nút trong lớp đầu ra của mình, một nút cho mỗi kết quả có thể có từ 0-199. Sau đó, chúng ta sử dụng dropout (0.2) được dùng để loại bỏ các đơn vị nhất định khỏi mô hình CNN, điều này sẽ giúp chúng ta ngăn chặn hiện tượng quá khớp (over fitting) trong mô hình và là một cách tiếp cận để điều chuẩn (regularization) trong mạng, giúp giảm sự phụ thuộc lẫn nhau giữa các nút của mạng. Sau đó, chúng ta sẽ sử dụng Batch Normalization trong lớp để chuẩn hóa và tái tỷ lệ dữ liệu. Và sau đó chúng ta có lớp đầu ra cuối cùng và thứ 2 là Dense (3, activation='softmax') có 3 chế độ cho mọi đầu ra từ 0-2 và hàm kích hoạt là softmax. Chúng ta cũng có thể lập trình mô hình tương tự bằng cách sử dụng hàm add, được dùng để lập trình từng lớp của mô hình CNN. Mã thay thế cho mô hình được xây dựng ở trên bằng hàm add:

Bước 4: Bây giờ chúng ta sẽ tạo dữ liệu cho mô hình và sau đó thực hiện huấn luyện.

```
train_datagen = ImageDataGenerator (rescale=1./255)

validation_datagen = ImageDataGenerator (rescale=1./255) image_size = 256

# Change the batchsize according to your system RAM train_batchsize = 20

val_batchsize = 20

train_generator = train_datagen.flow_from_directory ( 'dataset/train',

    target_size=(image_size, image_size), batch_size=train _
    batchsize, class_mode='categorical')

validation_generator = validation_datagen.flow_from_directory (

    'dataset/val',

    target_size=(image_size, image_size), batch_size=val _
    batchsize,

    class_mode='categorical', shuffle=True)
```

Output:

Found 1721 images belonging to 3 classes.

Found 431 images belonging to 3 classes

Ở đây chúng ta đang tạo bộ dữ liệu liên quan đến khoai tây và chia chúng thành các tập huấn luyện (training) và kiểm tra (testing) cho mô hình CNN của mình. Như bạn có thể thấy trong mã, chúng ta đã khai báo kích thước hình ảnh (image size) và kích thước lô (batch size) có thể thay đổi tùy theo nhu cầu của bộ dữ liệu và hệ thống.

Ở đây chúng ta đang tạo một hàm có tốc độ học (learning rate) sẽ thay đổi dựa trên các thay đổi của epoch (lượt huấn luyện) để chúng ta có thể nhận được kết quả tốt hơn từ mô hình đã huấn luyện. Lưu ý: Chúng ta cũng có thể giữ một tốc độ học không đổi cho tất cả các epoch. Đó hoàn toàn là mong muốn của chúng ta. Đôi khi nếu chúng ta không nhận được độ chính xác yêu cầu khi kiểm tra mô hình, chúng ta sẽ thay đổi tốc độ học.

```
def step_decay (epoch):
    if epoch<=6:
        return 1e-3
    elif epoch< $<=20$:
        return 1e-4
    else:
        return 1e-5
lr=step_decay
LearningRateScheduler (step_decay)
```

Ở đây chúng ta sẽ gọi lại mô hình của mình và bắt đầu huấn luyện mô hình bằng cách đặt các giá trị tốc độ học và trình tối ưu hóa (optimizers) để có kết quả tốt hơn mà bạn có thể thấy trong các output.

```
#define model (định nghĩa mô hình)
model=model_cnn ()
#calling the learning rate (gọi tốc độ học)
callbacks=[lr]
# Compile the model (Biên dịch mô hình)
model.compile (loss='categorical_crossentropy',
optimizer=Adam ($lr=0.0001$, beta $1=0.9$, beta $2=0.999$,
epsilon=1e-08, decay $=0.0?$, metrics=['acc'])
history=model.fit_generator (
train_generator,
steps_per_epoch=train_generator.samples/train_generator.
batch_size,
validation_data=validation_generator,
validation_steps=validation_generator.samples/validation_
```

```
epochs = 23,  
callbacks = callbacks)
```

Output:

(Phần output của quá trình huấn luyện Epoch 1-23 được tóm tắt) Epoch 1/23... Epoch 6/23... val loss: 0.0892 ... Epoch 16/23... (Bảng tóm tắt Epoch 17-23) Epoch 17/23... Epoch 18/23... Epoch 19/23... Epoch 20/23... Epoch 21/23... Epoch 22/23... Epoch 23/23...

Như bạn thấy, chúng ta đã huấn luyện mô hình của mình và đạt được độ chính xác (accuracy) là 99.42% và độ chính xác xác thực (validation accuracy) là 97.68%. Sau đó, chúng ta sẽ vẽ biểu đồ kết quả của mình bằng matplotlib.

Bước 5: Tổng kết Mô hình sau khi huấn luyện

```
# summarize history for accuracy (tổng kết lịch sử về độ chính xác)  
plt.plot(history.history['acc'])  
plt.plot(history.history['val  
acc'])  
plt.title('model accuracy')  
plt.ylabel('accuracy')  
plt.xlabel('epoch')  
plt.legend(['train', 'test'], loc='upper left')  
plt.show()
```

Output: as in Fig. 10.1 (như trong Hình 10.1)

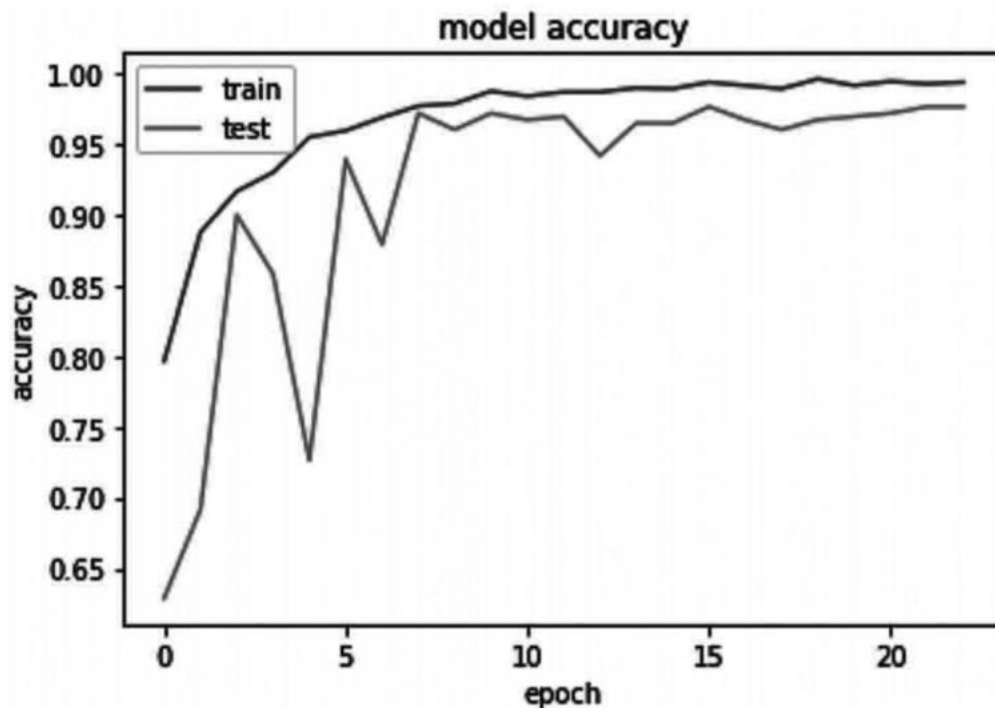


Fig. 10.1 Accuracy Representation of model

```
# summarize history for loss (tổng kết lịch sử về mất mát)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

Output: as in Fig. 10.2 (như trong Hình 10.2)

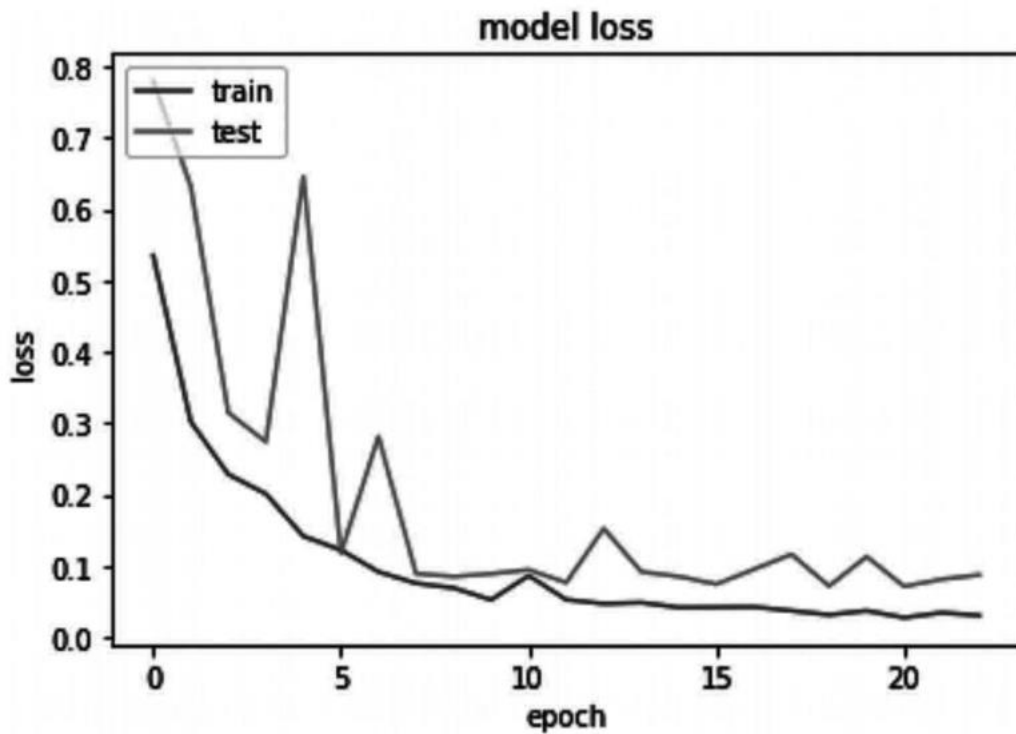


Fig. 10.2 Loss Representation of model

Ở đây chúng ta đã vẽ biểu đồ kết quả giữa độ chính xác của mô hình và mất mát của mô hình. Bạn có thể thấy trong các biểu đồ.

Bước 6: Lưu mô hình

```
model_json = model.to_json()
with open("model.json",
          "w") as json_file:
    json_file.write(model_json)
```

```
# serialize weights to HDF5 (tuyển tự hóa trọng số sang HDF5)
model.save_weights("model.h5")
print ("Model Saved")
```

output: Model Saved Trong bước này, chúng ta đang lưu mô hình đã huấn luyện để có thể sử dụng sau này. Bằng cách này, chúng ta không cần phải huấn luyện lại mô hình của mình nhiều lần cho bất kỳ dữ liệu đầu vào mới nào để kiểm tra kết quả của chúng.

Bước 7: Kiểm tra mô hình Ở đây chúng ta sẽ kiểm tra mô hình mà chúng ta đã lưu ở bước 5 trước đó. #importing the model (nhập mô hình)

```
from tensorflow.keras.models import model_from_json
json_file =
loaded_model =
model_from_json(json_file.read())
file.close()
loaded_model.load_weights('model.h5')
```

Đầu tiên, chúng ta tải mô hình đã huấn luyện của mình, mô hình này sẵn sàng để sử dụng cho việc phát hiện bệnh ở khoai tây và tình trạng khỏe mạnh của nó.

```
class dict_train_generator:
    indices = list(class dict.keys())
    class list
```

ở đây chúng ta đang tạo một danh sách cho tình trạng bệnh của cây liên quan đến các danh mục của nó class list=['early blight', 'healthy', 'late blight'] Ở đây chúng ta đang định nghĩa danh sách với tên bệnh để dự đoán khi cung cấp đầu vào.

```
import cv2
def preprocess(image):
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = cv2.resize(image, (256, 256))
    # create a simple mask image th the original one (tạo một hình ảnh mặt nạ đơn giản với hình ảnh gốc)
    mask = np.zeros(image.shape[:2], np.uint8)
    # specify the background and foreground model (chỉ định mô hình nền và tiền cảnh)
```

```

backgroundModel = np.zeros( (1,
65), np.float64)
foregroundModel = np.zeros( (1,
65), np.float64)
# define the Region of Interest (ROI) as the coordinates of the
rectangle (xác định Vùng quan tâm (ROI) là tọa độ của hình chữ nhật)
rectangle = (20, 20, 220, 220)
cv2.grabCut (image, mask, rectangle,
backgroundModel, foregroundModel,
1,cv2.GC_INIT_WITH_RECT)
mask2
image
image
np.where ( (mask == 2)|(mask == 0), 0, 1).astype ('uint8')
*
image mask2[:, :, np.newaxis]
image/255
return image

```

Ở đây chúng ta đã sử dụng Thị giác máy tính (Computer Vision) để cung cấp đầu vào cho mô hình đã huấn luyện.

```

def predict class (model,image path):
original image cv2.imread (image path)
image preprocess (original
tab = []
tab.append (image)
tab = np.array (tab)
h = np.expand
image)
dims (tab, axis=2)
h=model.predict (tab)
predicted class = np.argmax (h[0])
confidence = np.max (h[0])
# plot results (vẽ kết quả)
plt.figure (figsize $(4,4))$
plt.imshow (image)
plt.axis ('off')
plt.title('prediction:
conf '+str (confidence))
plt.show ()
'+class list [predicted class]+' with

```

ở đây chúng ta đang vẽ kết quả cho các hình ảnh dữ liệu đầu vào vào mô hình.

```
predict_class (loaded_model,'test/lateb.jpg') #image path is
provided for prediction (đường dẫn hình ảnh được cung cấp để dự đoán)
predict_class (loaded
```

Output:

```
model, 'test/lateb2.jpg')
prediction late blight with conf 0.9999802
prediction late blight with conf 0.99996245
```

(Các hình ảnh đầu ra được hiển thị trong tài liệu gốc)

```
predict_class (loaded_model,'test/earlyb.jpg')
predict_class (loaded_model,'test/earlybl.jpg')
```

Output: prediction late blight with conf 0.9517815 prediction late blight with conf 9.5648595

```
predict_class (loaded_model, 'test/healthy.jpg')
predict_class (loaded_model,'test/healthy1.jpg')
```

Output: prediction late blight with conf 0.5035014

Như bạn đã thấy trong dự án, cách một mô hình CNN được tạo ra để phát hiện bệnh cây trồng và cách chúng ta đã lưu mô hình để sử dụng trong tương lai. Bạn có thể tải xuống mô hình đã lưu từ liên kết này và có thể sử dụng trực tiếp để dự đoán. #link is

Tóm tắt

Phân loại bệnh cây trồng đã rất nổi bật trong nông nghiệp, điều này làm cho nó trở nên hữu ích hơn cho nông dân trong việc xác định và ngăn chặn bệnh lây lan giữa các cây trồng để có thể ngăn ngừa thiệt hại cho cây trồng. Như trong chương này, chúng ta đã sử dụng bộ dữ liệu plant village để xác định bệnh ở cây trồng bằng cách sử dụng các triệu chứng của nó trên lá cây bằng cách sử dụng học máy (machine learning), CNN và thị giác máy tính, là các lĩnh vực của trí tuệ nhân tạo. Dự án hiện tại đã được thực hiện trong chương này có thể được sử dụng trong thời gian thực để xác định bệnh ở cây khoai tây. Tương tự như vậy, chúng ta có thể thiết kế một hệ thống nhận dạng và xác định bệnh ở thực vật cho mọi loại cây trồng, điều này có thể mang lại lợi ích cho nông dân trong quá trình canh tác của họ.